

Preliminary Project Proposal

Hospital Readmission Risk Predictor: A Phased MLOps Build

Basics

Preliminary project title: Hospital Readmission Risk Predictor — A Phased MLOps Build

Team member: Angeline Setiawan (working individually)

Roles this week: Angeline Setiawan is Team Lead, Documentation Lead, and Communication Lead.

Team name: Epsilon

GitHub repo URL:

<https://github.com/AngieLie/AngelineSetiawan-hospital-readmission-risk-predictor>

Background & Question

Research question: Can a patient's risk of being readmitted to the hospital within 30 days of discharge be predicted from clinical encounter data, and can that prediction be delivered through a live, self-updating machine learning system rather than a one-off model?

Why this question matters: Hospitals in the U.S. are financially penalized under CMS's Hospital Readmissions Reduction Program when their 30-day readmission rates are too high. A reliable, deployable risk model gives hospitals a way to flag high-risk patients before discharge so that care teams can intervene early (e.g., follow-up calls, care coordination, medication review) rather than reacting after a readmission has already occurred.

Real-world niche addressed: Most classroom-style readmission models stop at reporting an accuracy score. This project instead treats the prediction as part of a working system — one that logs experiments, promotes the best model, serves predictions through an API, and (in the stretch phase) retrains and redeploys itself automatically. That operational layer is the niche: proving a model works is different from proving it can be run responsibly in production.

Primary stakeholder / audience: Hospital administrators and care management or utilization-review teams who are accountable for readmission rates and would act on the model's risk scores. A secondary audience is health-system data/ML engineering teams interested in the MLOps pattern itself.

Prior work: The Diabetes 130-US Hospitals dataset is a well-established benchmark in the healthcare ML literature, and 30-day readmission prediction has been studied extensively using logistic regression, random forests, and gradient boosting. This project builds on that prior work rather than trying to out-perform it on accuracy alone; the point of differentiation is the surrounding system (experiment tracking, model registry, serving, and conditional automated retraining), which most prior academic treatments of this dataset do not include.

Hypothesis: Patients with more complex or intensive care histories — longer hospital stays, more prior inpatient/emergency visits, and a higher number of medications — are more likely to be readmitted within 30 days, because these factors reflect greater underlying disease severity and more complicated post-discharge care needs.

Prediction: A supervised classification model trained on this dataset will identify patients at elevated 30-day readmission risk with meaningfully better performance than a naive baseline (e.g., always predicting the majority class), and utilization-related features such as number of inpatient visits, time in hospital, and number of medications will rank among the most important predictors.

Data & Analysis

Dataset:

<https://archive.ics.uci.edu/dataset/296/diabetes+130-us+hospitals+for+years+1999-2008>

The UCI Diabetes 130-US Hospitals dataset covers ten years (1999–2008) of clinical care at 130 U.S. hospitals, with approximately 101,766 patient encounters and more than 50 features per encounter. It is licensed CC BY 4.0, requires no login, and downloads directly as a CSV.

Citation: Clore, J., et al. "Diabetes 130-US Hospitals for Years 1999-2008." UCI Machine Learning Repository, 2014, <https://doi.org/10.24432/C5230J>.

Why this data fits: The dataset is large enough to support both a supervised classifier and two unsupervised analyses (a requirement of the assignment), and it includes a genuine readmission outcome label along with rich demographic, diagnostic, medication, and prior-utilization fields needed to test the hypothesis above.

How the data will be obtained: Direct CSV download from the UCI Machine Learning Repository — no API key, scraping, or database access required.

Planned Variables

- Target (outcome) variable: A binary 30-day readmission indicator, collapsed from the dataset's native three-class label (<30 days, >30 days, No readmission) into 1 (<30 days) vs. 0 (everything else).
- Key predictor variables: age bracket, time in hospital, number of medications, number of prior inpatient/outpatient/emergency visits, primary/secondary diagnosis codes, number of lab procedures, and admission/discharge type.

Preliminary Analysis Plan

Baseline approach: Logistic regression as the interpretable baseline classifier.

Other approaches under consideration: Random forest and XGBoost for the main supervised classifier; K-means or hierarchical clustering as the first unsupervised model, to group patients into risk profiles based on utilization and demographic features; and PCA (or t-SNE) as the

second unsupervised model, to reduce the 50+ features to a handful of components for visualization and, potentially, as engineered inputs to the supervised model.

Anticipated preprocessing: Handling the dataset's missing-value placeholders, collapsing the three-class target into a binary label, encoding high-cardinality categorical fields (especially ICD-9 diagnosis codes), and addressing class imbalance — since 30-day readmissions are a minority outcome — likely through class weighting or resampling.

Anticipated challenges / pitfalls: Class imbalance could inflate accuracy while hiding poor recall on the positive (readmitted) class, so evaluation will lean on AUC-ROC and recall/precision rather than raw accuracy. High-cardinality diagnosis codes risk overfitting if not grouped or encoded carefully. Care must also be taken to exclude any features only known after discharge, to avoid leaking information the model wouldn't have at prediction time. On the systems side, environment setup for the orchestration tooling (Airflow) is the most likely place to lose time and has been budgeted extra buffer for.

Determining whether the research question is answered: Success means the trained classifier outperforms the naive baseline by a meaningful margin on AUC-ROC and recall for the readmitted class, and that the model can be served through a live prediction endpoint — demonstrating both predictive value and deployability.

Evaluating the hypothesis: The hypothesis will be considered supported if feature-importance or SHAP analysis shows utilization/complexity variables (time in hospital, number of prior visits, number of medications) ranking among the top predictors, consistent with the reasoning that care complexity drives readmission risk.

Technical Details

Programming language: Python.

Anticipated tools: pandas and scikit-learn for data processing and baseline modeling; XGBoost for the primary supervised classifier; MLflow for experiment tracking and model registry; FastAPI to serve predictions from whichever model is tagged "Production"; Airflow with docker-compose (stretch phase) to orchestrate scheduled retraining, evaluation, and conditional promotion; JupyterLab as the development environment.

GitHub repository:

<https://github.com/AngieLie/AngelineSetiawan-hospital-readmission-risk-predictor>

Planned Phasing (for context)

Phase 1 (required, weeks 1–3): Clean and engineer features on the dataset; run clustering and PCA as exploratory unsupervised analysis; train the supervised classifier as a reusable, callable Python script; log runs to MLflow; manually promote the best run to "Production"; and build a

FastAPI endpoint that serves predictions from the current Production model. This alone is a complete, defensible capstone.

Phase 2 (stretch, weeks 4–6): Wrap the Phase 1 training script in an Airflow DAG that retrains on a schedule, evaluates the new model against the current Production model, and conditionally promotes it only if it wins — turning the project from "a trained model" into a self-updating ML system. FastAPI requires no changes, since it already reads whatever model is tagged Production.